☕ Coffee Quality Prediction

Machine Learning – Training & Evaluation Pipeline

📋 Project Overview

بناء نموذج Machine Learning يتوقع جودة القهوة باستخدام عدة خوارزميات Regression، ثم مقارنة أدائها واختيار أفضل نموذج وحفظه للاستخدام داخل التطبيق.

1. Problem Definition

- **Learning Type:** Supervised Learning
- **Problem Type:** Regression
- **Target:** `Total.Cup.Points`

Feature	Description
`Species`	نوع القهوة
`Country.of.Origin`	بلد المنشأ
`Variety`	سلالة القهوة
`Processing.Method`	طريقة المعالجة
`Total.Cup.Points`	درجة الجودة (Target)

2. Import Libraries

```
import json
import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

from sklearn.model_selection import (
    train_test_split,
    KFold,
    cross_val_score,
    RandomizedSearchCV
)
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

from sklearn.linear_model import LinearRegression, Ridge
```

```
from sklearn.ensemble import (
    RandomForestRegressor,
    ExtraTreesRegressor,
    GradientBoostingRegressor
)

from sklearn.dummy import DummyRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

try:
    from xgboost import XGBRegressor
    HAS_XGBOOST = True
except ImportError:
    HAS_XGBOOST = False

warnings.filterwarnings("ignore")
```

الفكرة: استيراد المكتبات الخاصة بقراءة البيانات، تجهيزها، تدريب النماذج، تقييمها، وحفظ النتائج. مكتبة XGBoost اختيارية.

3. Project Configuration

```
RANDOM_STATE = 42
TEST_SIZE = 0.20
CV_FOLDS = 5

PROJECT_ROOT = Path(__file__).resolve().parent

DATASET_PATH = (
    PROJECT_ROOT / "datasets" / "merged_data_cleaned.csv"
)

MODEL_DIR = PROJECT_ROOT / "backend" / "models"
MODEL_DIR.mkdir(parents=True, exist_ok=True)

FEATURE_COLS = [
    "Species",
    "Country.of.Origin",
    "Variety",
    "Processing.Method"
]

TARGET_COL = "Total.Cup.Points"
```

الفكرة: تحديد إعدادات المشروع، مسارات الملفات، أعمدة الإدخال `Features`، والعمود المطلوب التنبؤ به `Target`.

Setting	Purpose
`RANDOM_STATE`	تثبيت العشوائية
`TEST_SIZE`	تخصيص 20% للاختبار
`CV_FOLDS`	استخدام 5 أجزاء في Cross Validation
`FEATURE_COLS`	أعمدة الإدخال
`TARGET_COL`	القيمة المطلوب توقعها

4. Load and Validate Dataset

```
def load_dataset(path: Path) -> pd.DataFrame:

    if not path.exists():
        raise FileNotFoundError(
            f"Dataset not found: {path}"
        )

    df = pd.read_csv(path)

    required_columns = FEATURE_COLS + [TARGET_COL]

    missing_columns = [
        col for col in required_columns
        if col not in df.columns
    ]

    if missing_columns:
        raise ValueError(
            f"Missing columns: {missing_columns}"
        )

    df = df.dropna(subset=[TARGET_COL])

    print(f"Dataset shape: {df.shape}")

    return df
```

الفكرة: تحميل ملف CSV، التأكد من وجود الأعمدة المطلوبة، وحذف الصفوف التي لا تحتوي على قيمة Target؛ لأن النموذج يحتاج إلى إجابة صحيحة ليتعلم منها.

5. Data Preprocessing

```
def get_preprocessor():

    categorical_transformer = Pipeline(
        steps=[
            (
                "imputer",
                SimpleImputer(
```

```
                    strategy="constant",
                    fill_value="Unknown"
                )
            ),
            (
                "onehot",
                OneHotEncoder(
                    handle_unknown="ignore",
                    sparse_output=False
                )
            )
        ]
    )

    preprocessor = ColumnTransformer(
        transformers=[
            (
                "cat",
                categorical_transformer,
                FEATURE_COLS
            )
        ]
    )

    return preprocessor
```

الفكرة: تجهيز البيانات النصية قبل إدخالها إلى النموذج.

- `SimpleImputer` : استبدال القيم المفقودة بـ `Unknown` .
- `OneHotEncoder` : تحويل القيم النصية إلى أعمدة رقمية.
- `ColumnTransformer` : تطبيق المعالجة على الأعمدة المحددة.
- `handle_unknown="ignore"` : التعامل مع القيم الجديدة التي لم تظهر أثناء التدريب.

6. Train / Test Split

```
df = load_dataset(DATASET_PATH)

X = df[FEATURE_COLS]
y = df[TARGET_COL]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

الفكرة: تقسيم البيانات إلى:

- **80% Training:** لتدريب النموذج.
- **20% Testing:** لاختبار النموذج على بيانات لم يرها أثناء التدريب.

7. Baseline Model

```
baseline = DummyRegressor(strategy="mean")

baseline.fit(X_train, y_train)

baseline_pred = baseline.predict(X_test)

baseline_mae = mean_absolute_error(
    y_test,
    baseline_pred
)

print("Baseline MAE:", baseline_mae)
```

الفكرة: إنشاء نموذج بسيط يتوقع متوسط درجات الجودة دائمًا. نستخدمه كمرجع لمعرفة هل نماذج Machine Learning تحقق تحسنًا حقيقيًا أم لا.

8. Define Models

```
models = {
    "Linear Regression": LinearRegression(),

    "Ridge Regression": Ridge(
        random_state=RANDOM_STATE
    ),

    "Random Forest": RandomForestRegressor(
        random_state=RANDOM_STATE
    ),

    "Extra Trees": ExtraTreesRegressor(
        random_state=RANDOM_STATE
    ),

    "Gradient Boosting": GradientBoostingRegressor(
        random_state=RANDOM_STATE
    )
}

if HAS_XGBOOST:
    models["XGBoost"] = XGBRegressor(
        random_state=RANDOM_STATE,
        verbosity=0
    )
```

الفكرة: تجربة عدة خوارزميات Regression ومقارنة أدائها بدل اختيار نموذج بشكل عشوائي.

Model	Main Idea
Linear Regression	تعلم العلاقات الخطية
Ridge	Linear Regression مع Regularization
Random Forest	مجموعة من Decision Trees
Extra Trees	أشجار بقدر أكبر من العشوائية
Gradient Boosting	نماذج متتابعة لتصحيح الأخطاء
XGBoost	خوارزمية Boosting متقدمة

9. Cross Validation

```
kf = KFold(
    n_splits=CV_FOLDS,
    shuffle=True,
    random_state=RANDOM_STATE
)

comparison_results = []

for name, model in models.items():

    pipeline = Pipeline(
        steps=[
            ("preprocessor", get_preprocessor()),
            ("regressor", model)
        ]
    )

    cv_scores = cross_val_score(
        pipeline,
        X_train,
        y_train,
        cv=kf,
        scoring="neg_mean_absolute_error"
    )

    comparison_results.append({
        "Model": name,
        "CV_MAE_Mean": -cv_scores.mean(),
        "CV_MAE_Std": cv_scores.std()
    })
```

الفكرة: استخدام 5-Fold Cross Validation لتقييم كل نموذج بصورة أكثر استقرارًا.

في كل تجربة:

- 4 أجزاء للتدريب.
- جزء واحد للتقييم.

تتكرر العملية 5 مرات، ويُستخدم كل جزء مرة واحدة للتقييم.

لماذا Pipeline؟

لضمان تنفيذ Preprocessing داخل كل Fold، وتجنب Data Leakage أثناء التقييم.

نستخدم `neg_mean_absolute_error` لأن Scikit-learn ترجع قيمة MAE سالبة عند استخدام نظام Scoring. لذلك نعكس الإشارة.

10. Model Comparison

```
comp_df = pd.DataFrame(
    comparison_results
).sort_values(
    by="CV_MAE_Mean"
)

comp_df.to_csv(
    MODEL_DIR / "model_comparison.csv",
    index=False
)

best_model_name = comp_df.iloc[0]["Model"]

print("Best Model:", best_model_name)
```

الفكرة: ترتيب النماذج حسب متوسط MAE واختيار النموذج صاحب أقل خطأ.

يتم حفظ نتائج المقارنة في `model_comparison.csv` .

11. Hyperparameter Tuning

```
base_pipeline = Pipeline(
    steps=[
        ("preprocessor", get_preprocessor()),
        ("regressor", models[best_model_name])
    ]
)

param_distributions = {}

if best_model_name in ["Random Forest", "Extra Trees"]:

    param_distributions = {
        "regressor__n_estimators": [50, 100, 200],
        "regressor__max_depth": [None, 10, 20],
        "regressor__min_samples_split": [2, 5, 10]
    }

elif best_model_name == "Gradient Boosting":
```

```
    param_distributions = {
        "regressor__n_estimators": [50, 100, 200],
        "regressor__learning_rate": [0.01, 0.05, 0.1],
        "regressor__max_depth": [3, 5, 7]
    }

elif best_model_name == "Ridge Regression":

    param_distributions = {
        "regressor__alpha": [0.1, 1.0, 10.0, 100.0]
    }

elif best_model_name == "XGBoost":

    param_distributions = {
        "regressor__n_estimators": [50, 100, 200],
        "regressor__learning_rate": [0.01, 0.05, 0.1],
        "regressor__max_depth": [3, 5, 7]
    }
```

الفكرة: تحديد مجموعات مختلفة من Hyperparameters لتحسين أداء النموذج الأفضل.

Parameter	Meaning
`n_estimators`	عدد الأشجار
`max_depth`	أقصى عمق للشجرة
`min_samples_split`	أقل عدد عينات لتقسيم Node
`learning_rate`	معدل التعلم
`alpha`	قوة Regularization في Ridge

12. RandomizedSearchCV

```
if param_distributions:

    search = RandomizedSearchCV(
        base_pipeline,
        param_distributions=param_distributions,
        n_iter=10,
        cv=kf,
        scoring="neg_mean_absolute_error",
        random_state=RANDOM_STATE,
        n_jobs=-1
    )

    search.fit(X_train, y_train)

    final_model = search.best_estimator_
    best_params = search.best_params_
```

```
else:

    final_model = base_pipeline
    final_model.fit(X_train, y_train)

    best_params = "Default Parameters"
```

الفكرة: تجربة 10 مجموعات من الإعدادات واختيار الأفضل باستخدام Cross Validation.

- `n_iter=10` : عدد التجارب.
- `cv=kf` : استخدام 5-Fold.
- `n_jobs=-1` : استخدام جميع أنوية المعالج المتاحة.
- `best_estimator_` : أفضل Pipeline.
- `best_params_` : أفضل إعدادات.

13. Final Model Evaluation

```
y_pred = final_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

print("Best Model:", best_model_name)
print("Test MAE:", mae)
print("Baseline MAE:", baseline_mae)
print("Test RMSE:", rmse)
print("Test R2:", r2)
```

الفكرة: اختبار النموذج النهائي على بيانات الاختبار التي لم تُستخدم في التدريب أو اختيار الإعدادات.

Metric	Meaning	Better
MAE	متوسط الخطأ المطلق	Lower
RMSE	يتأثر بالأخطاء الكبيرة	Lower
R^2	قدرة النموذج على تفسير التغير في النتائج	Higher

14. Save Trained Model

```
joblib.dump(
    final_model,
    MODEL_DIR / "coffee_model.joblib"
)
```

الفكرة: حفظ النموذج النهائي كاملًا، بما فيه خطوات Preprocessing والـ Encoder والنموذج المدرب، لاستخدامه لاحقًا داخل Backend.

15. Save Evaluation Metrics

```
metrics_data = {
    "best_model": best_model_name,
    "test_mae": round(float(mae), 4),
    "baseline_mae": round(float(baseline_mae), 4),
    "test_rmse": round(float(rmse), 4),
    "test_r2": round(float(r2), 4)
}

with open(
    MODEL_DIR / "evaluation_metrics.json",
    "w",
    encoding="utf-8"
) as f:

    json.dump(
        metrics_data,
        f,
        indent=4,
        ensure_ascii=False
    )
```

الفكرة: حفظ نتائج التقييم في ملف JSON لمراجعتها أو استخدامها داخل التطبيق.

16. Save Training Metadata

```
metadata = {
    "feature_columns": FEATURE_COLS,
    "target_column": TARGET_COL,
    "test_size": TEST_SIZE,
    "cv_folds": CV_FOLDS,
    "random_state": RANDOM_STATE,
    "best_hyperparameters": str(best_params)
}

with open(
    MODEL_DIR / "training_metadata.json",
    "w",
    encoding="utf-8"
) as f:

    json.dump(
        metadata,
        f,
        indent=4,
        ensure_ascii=False
    )
```

الفكرة: حفظ إعدادات التدريب والـ Features والـ Target وأفضل Hyperparameters، حتى نعرف كيف تم تدريب النموذج.

17. Main Entry Point

```
if __name__ == "__main__":
    train()
```

الفكرة: تشغيل دالة التدريب الرئيسية عند تشغيل الملف مباشرة.

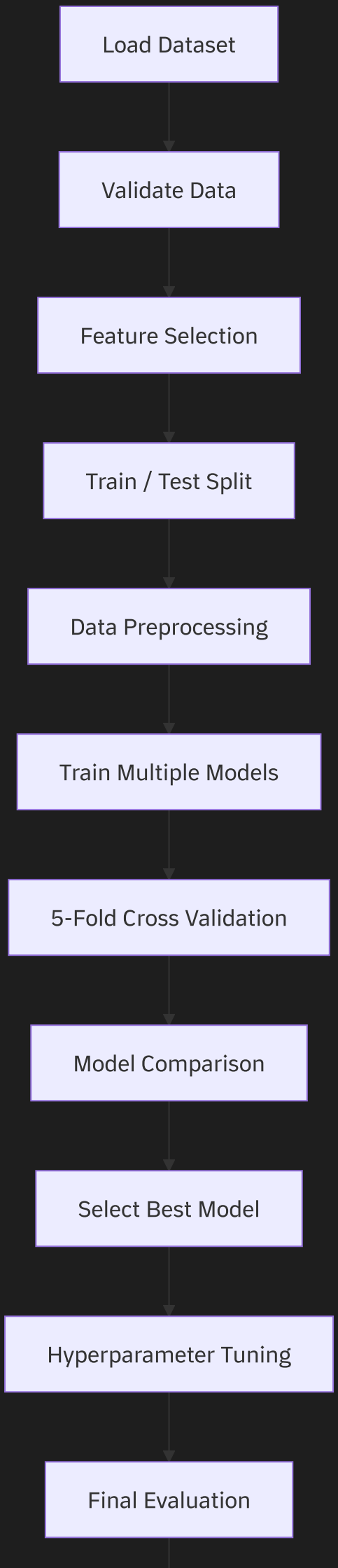
```
python train.py
```

18. Output Structure

```
Project/
│
├── datasets/
│   └── merged_data_cleaned.csv
│
├── backend/
│   └── models/
│       ├── coffee_model.joblib
│       ├── model_comparison.csv
│       ├── evaluation_metrics.json
│       └── training_metadata.json
│
└── train.py
```

File	Purpose
`coffee_model.joblib`	النموذج النهائي
`model_comparison.csv`	مقارنة النماذج
`evaluation_metrics.json`	نتائج التقييم
`training_metadata.json`	إعدادات التدريب

19. Complete Workflow


Load Dataset
Validate Data
Feature Selection
Train / Test Split
Data Preprocessing
Train Multiple Models
5-Fold Cross Validation
Model Comparison
Select Best Model
Hyperparameter Tuning
Final Evaluation

Save Model and Metrics

Key Takeaways

Concept	Meaning
Supervised Learning	التعلم من بيانات لها إجابات معروفة
Regression	التنبؤ بقيمة رقمية
Features	المدخلات
Target	القيمة المطلوب توقعها
Preprocessing	تجهيز البيانات
Imputation	معالجة القيم المفقودة
One-Hot Encoding	تحويل النصوص إلى أرقام
Pipeline	ربط المعالجة بالنموذج
Baseline	نموذج مرجعي للمقارنة
Cross Validation	تقييم النموذج على تقسيمات متعددة
Hyperparameter Tuning	تحسين إعدادات النموذج
Overfitting	حفظ بيانات التدريب وضعف التعميم

⚠ Important Notes

- الـ Hyperparameter Tuning يتم لبعض النماذج فقط حسب الكود.
- `Linear Regression` لا يحتوي على مساحة بحث مخصصة، ويُستخدم بإعداداته الافتراضية.
- `Extra Trees` مشمول في مساحة البحث الخاصة بـ Random Forest.
- `sparse_output=False` يتطلب إصدارًا حديثًا من Scikit-learn (1.2 أو أحدث).

Final Summary

الـ Pipeline بتنّفذ دورة Machine Learning كاملة:

1. تحميل البيانات والتحقق منها.
2. تجهيز البيانات وتحويل النصوص إلى أرقام.
3. تقسيم البيانات إلى Training وTesting.
4. تدريب ومقارنة عدة Regression Models.
5. اختيار أفضل نموذج وتحسين إعداداته.
6. تقييم النموذج النهائي.
7. حفظ النموذج والنتائج لاستخدامها في التطبيق.